

There are three basic functions in the DataStore API: specialized tables, distributed tables, and associations that are split into five separate APIs (User Preference, Object Data Definition, Object Data Access, Association Data Definition, and Association Data Access).

According to Facebook, it can handle millions of records with little or no performance degradation, something we can't be sure of about data stored on local servers. Because indexing tables in a distributed environment won't necessarily provide a performance boost, Facebook provides a specialized table of users that is optimized to provide performance for fast lookups (such as indexes) with the help of the associations component of this API. This mechanism has been implemented to provide fast lookups without centralized indexes or parallel queries. The user preferences for the API are a list of 128-character strings, for which you can store up to 201 for each user (numbered 0-200). Access to the getters/setters methods are accessed through getters and setters in the REST API (**facebook.data.setUserPreference** and **facebook.data.getUserPreferences**).

Data objects can be created with **facebook.createObjectType**. These objects take a name and contain a set of object properties like data types. However, you don't have the same type of control over the data types with this API as you do with normal RDBMS because you are limited to integers, strings (less than 256 characters), and text blobs (with a maximum of 64 KB).

You create, read, update, and delete through the Object Data Access API after you have defined your objects and object types. These go like **facebook.data.createObject**. But, you first need to define the relationship between objects in the **facebook.defineAssociation** call to work with the associations between objects. There are two basic types of associations: one-way, symmetric associations and asymmetric associations. The asymmetric association is like a many-to-many join and a symmetric association is like a one-to-many join. You can then create the actual associations with the help of the Association Data Access API which contains methods required to create, remove, and retrieve these associations and retrieve the object details from the data contained in the data definition.

Example:

```
<?php
$createObject      =    $facebook->api_client->data_
createObjectType("wallpost");
$uid = $facebook->api_client->data_defineObjectProperty
("wallpost", "uid", 1);
```